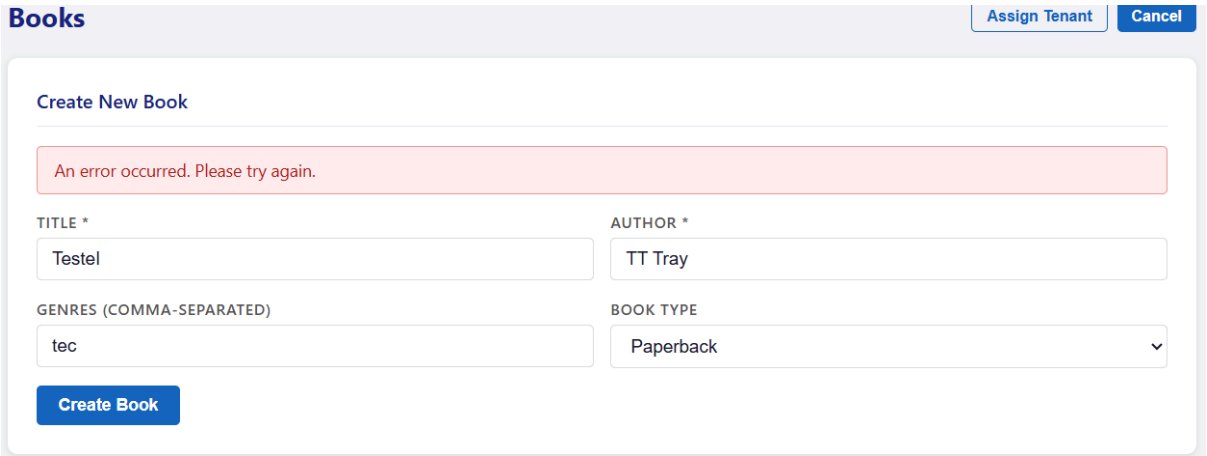
Relatório de Incidente de Integração e Segurança - Goledger Challenge

ID do Chamado: BUG-001 | **Prioridade:** Alta (Blocker) **Módulo:** Gestão de Livros (Front-end / Integração API) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)




```
export async function createBook(data: CreateBookData): Promise<Book> {
  const res = await fetch(`${API_BASE}/books`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(data),
  });
  const body = await res.json();
  if (!res.ok) throw new Error(body.error ?? 'Failed to create book');
  return body as Book;
}
```

1. Descrição do Problema Durante a validação do fluxo de criação de ativos, identificou-se uma falha impeditiva no módulo de Gestão de Livros. Mesmo após o preenchimento correto de todos os parâmetros exigidos pelo formulário, o sistema bloqueia a criação do livro. A interface de usuário (UI) não realiza o tratamento da exceção, exibindo um alerta genérico ou omitindo o motivo da falha. A análise de tráfego de rede (Network DevTools) revelou que o servidor está rejeitando a requisição com o código de status HTTP `401 Unauthorized`.

2. Passos para Reproduzir (Reproducibility)

- **Acesso:** Autentique-se na plataforma Web utilizando credenciais válidas de Administrador (ex: `admin` / `admin123`).
- **Navegação:** Acesse a funcionalidade de "Novo Livro" ou "Criar Livro".
- **Preenchimento:** Insira dados válidos em todos os campos obrigatórios (Título e Autor).
- **Ação:** Acione o botão de submissão/salvar.

- **Observação Visual:** Note que o livro não é listado e a tela apresenta feedback de erro genérico.
- **Análise de Rede:** Abra a aba *Network* no navegador e verifique a requisição `POST` disparada, confirmando o recebimento do status `401 Unauthorized`.

3. Análise da Causa Raiz O erro ocorre na camada de integração do Front-end. A função responsável por realizar a chamada (Fetch/Axios) para o endpoint de criação de livros não está anexando a credencial de segurança do usuário. A requisição está sendo enviada sem o cabeçalho (`header`) obrigatório de autorização. Como a rota no Back-end é protegida, ela rejeita imediatamente a requisição "anônima" com um 401.

4. Impacto Operacional

- **Funcionalidade:** Trata-se de um bloqueador crítico (*blocker*), pois impede os administradores de alimentarem o catálogo do sistema com novos ativos.
- **Usabilidade (UX):** O tratamento inadequado do erro 401 na interface deixa o usuário "cego" e frustrado, sem saber se cometeu um erro de digitação ou se a sessão expirou.

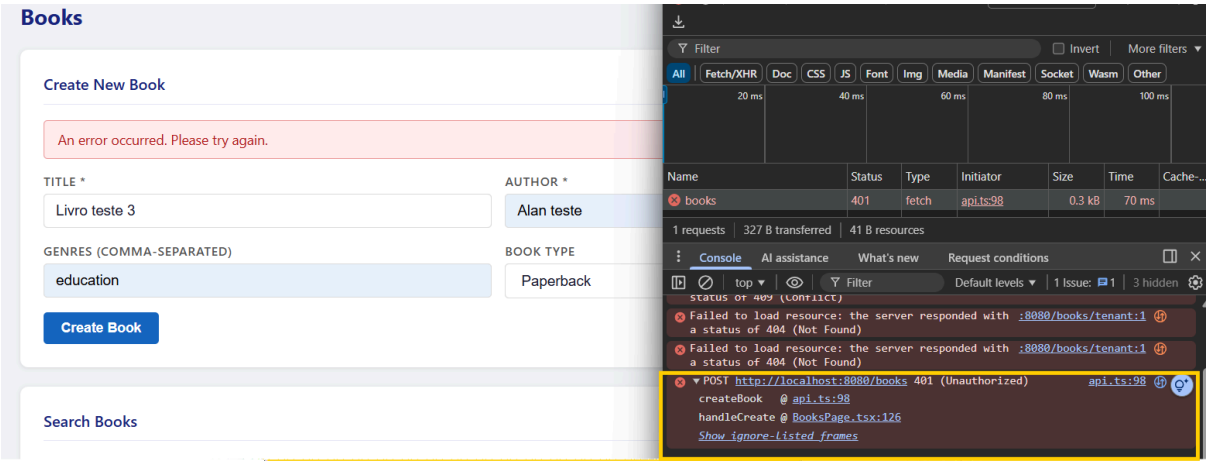
5. Recomendações Técnicas e Solução Para a resolução deste incidente, são necessárias duas atuações no código Front-end:

- **Injeção de Credencial:** Interceptar a requisição HTTP e injetar dinamicamente o Token JWT salvo na sessão. Adicionar ao payload o parâmetro: `headers: { 'Authorization': \`Bearer ${getToken()}` }`.
- **Tratamento de Exceção (UX):** Implementar um bloco `catch` estruturado que, ao identificar um retorno 401 da API, exiba um alerta amigável de "Sessão expirada" e force o redirecionamento do usuário para a tela de Login.

6. Evidência de Fechamento (Expectativa Pós-Correção) Após a inclusão do cabeçalho de autorização, a mesma requisição deverá ser validada pelo Back-end, retornando o status HTTP `201 Created`. A interface deverá exibir um alerta claro de "Livro criado com sucesso" e atualizar a lista de ativos imediatamente.

Relatório de Incidente de Interface e Mensageria - Goledger Challenge

ID do Chamado: BUG-002 | **Prioridade:** Média (Impacto na Experiência do Usuário)
Módulo: Global / Tratamento de Erros (Front-end) **Plataforma:** Aplicação Web (React/Vite)
Relator: Vitória (QA)



1. Descrição do Problema Identificou-se que a aplicação carece de um sistema de mensageria detalhado para falhas de comunicação com a API. Ao ocorrer uma interrupção no fluxo (seja por erro de autenticação, validação ou queda de servidor), o Front-end exibe um alerta genérico ou simplesmente não fornece feedback visual ao usuário. A ausência de mensagens de erro descritivas impede que o usuário entenda se o problema é de preenchimento, de permissão ou uma falha técnica momentânea.

2. Passos para Reproduzir (Reproducibility)

- **Simulação de Falha:** Tente realizar qualquer ação que gere um erro de resposta do servidor (ex: cadastrar um livro sem o token de autorização ou buscar uma biblioteca inexistente).
- **Interação:** Clique no botão de ação e observe o feedback da tela.
- **Observação Visual:** Note que a interface apresenta apenas um alerta padrão (ou uma caixa de diálogo vazia/genérica) sem especificar o código do erro ou a orientação para correção.
- **Verificação de Log:** Ao abrir o console do navegador (F12), o erro técnico está presente (ex: 401, 400 ou 500), mas essa informação não é traduzida para o componente de UI.

3. Análise da Causa Raiz A aplicação está utilizando um bloco de tratamento de erros (`try/catch` ou `interceptors`) excessivamente simplificado. O código captura a falha, mas não realiza o *parsing* do corpo da resposta (JSON) enviada pelo Back-end, que geralmente contém o campo `"error"` ou `"message"`. Consequentemente, o componente de alerta não recebe uma string dinâmica para exibir.

4. Impacto Operacional

- **Suporte Técnico:** Aumenta o tempo de resolução de problemas, pois o usuário não consegue reportar o que deu errado com precisão.
- **Retenção e UX:** Gera uma percepção de instabilidade e falta de polimento no produto, reduzindo a confiança do usuário final.

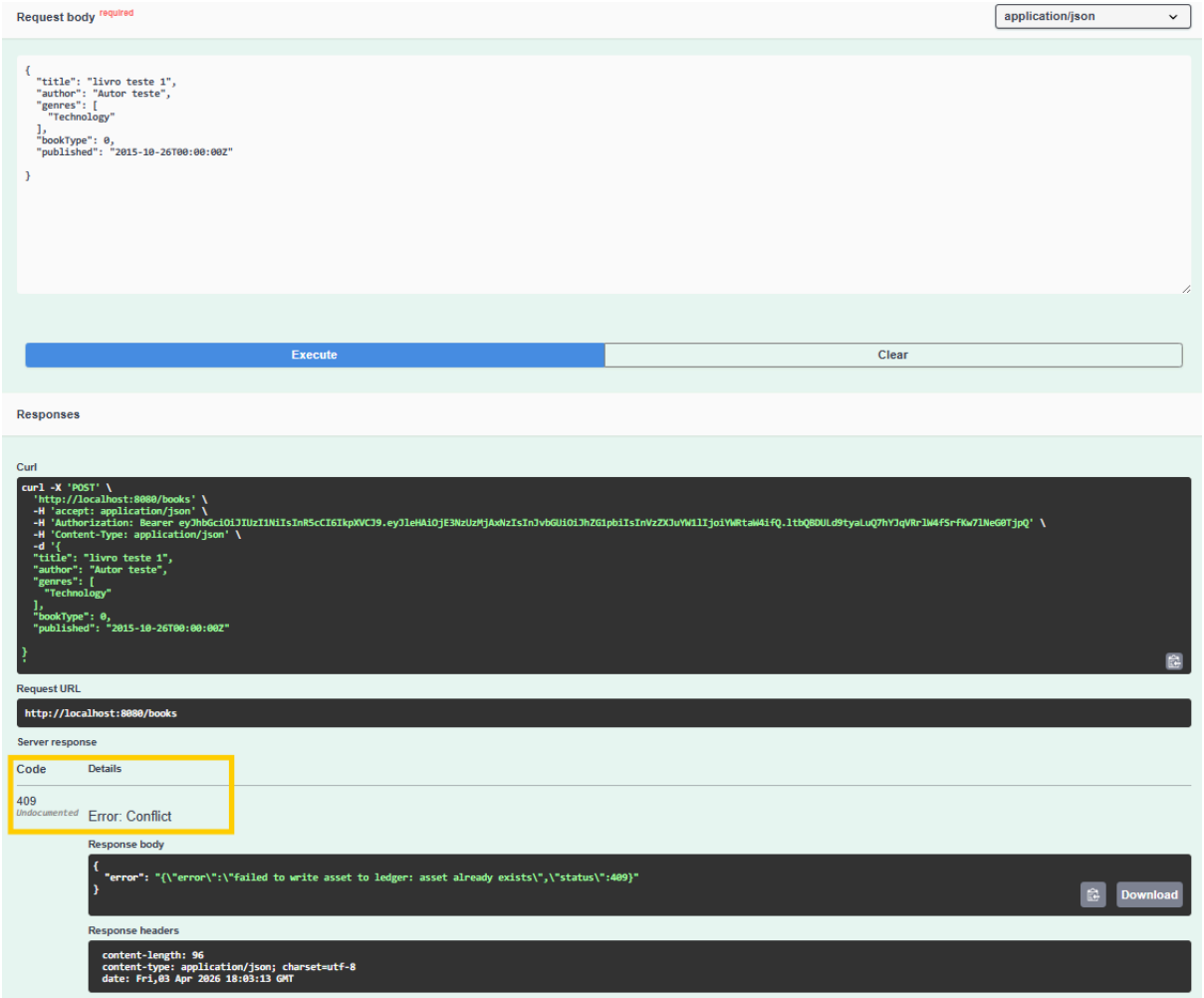
5. Recomendações Técnicas e Solução

- **Mapeamento de Status HTTP:** Implementar um switch-case no serviço de API do Front-end para tratar os códigos mais comuns:
    - **401:** "Sessão expirada. Por favor, faça login novamente."
    - **400:** Exibir a mensagem específica de validação vinda da API.
    - **404:** "Recurso não encontrado."
    - **500:** "Erro interno no servidor. Tente novamente mais tarde."
- **Componente de Toast/Alert:** Integrar uma biblioteca de notificações (como `react-toastify` ou similar) que suporte títulos e descrições dinâmicas baseadas no retorno do servidor.

6. Evidência de Fechamento (Expectativa Pós-Correção) Ao ocorrer uma falha, o usuário deve visualizar um componente de alerta contendo o motivo real do erro (ex: "Erro 401: Não autorizado") ou uma orientação clara de como proceder, garantindo autonomia na navegação.

Relatório de Incidente de Documentação (API Contract) - Goledger Challenge

ID do Chamado: BUG-003 | **Prioridade:** Baixa (Inconsistência de Documentação) **Módulo:** API / Swagger (Books Endpoint) **Plataforma:** Backend / Documentação OpenAPI **Relator:** Vitória (QA)



1. Descrição do Problema Identificou-se uma divergência entre o comportamento real da API e a documentação técnica (Swagger) no endpoint de criação de livros. Ao tentar cadastrar um livro com credenciais (título/autor) já existentes no banco de dados, a API reage corretamente bloqueando a duplicidade, porém, utiliza um código de status HTTP diferente do documentado. Enquanto a documentação prevê o retorno de `400 Bad Request`, o servidor entrega o status `409 Conflict`.

2. Passos para Reproduzir (Reproducibility)

- **Acesso:** Acesse a interface do Swagger UI da aplicação.
- **Preparação:** Realize a criação de um livro (ex: Título: "Dom Casmurro", Autor: "Machado de Assis").

- **Ação de Teste:** Tente realizar um novo `POST` com exatamente os mesmos dados do livro anterior.
- **Verificação de Resposta:** Observe o campo *Response Code* no Swagger.
- **Constatação:** O sistema retorna `409 Conflict`, mas na seção "Responses" da documentação do endpoint, o erro de validação está mapeado apenas como `400`.

3. Análise da Causa Raiz Houve uma falha na sincronização entre a implementação da lógica de exceção no código (onde o desenvolvedor optou, acertadamente do ponto de vista REST, pelo status `409`) e a anotação/decorador da documentação OpenAPI, que não foi atualizada para refletir este comportamento específico de conflito de dados.

4. Impacto Operacional

- **Integração:** Desenvolvedores de Front-end ou de sistemas externos que programarem suas aplicações baseados estritamente na documentação (ex: esperando um 400 para tratar erros) não conseguirão capturar a exceção de duplicidade de forma correta, pois o sistema enviará um código (409) não previsto.
- **Confiabilidade:** Reduz a precisão da documentação técnica, que deve ser a "única fonte da verdade" para o consumo da API.

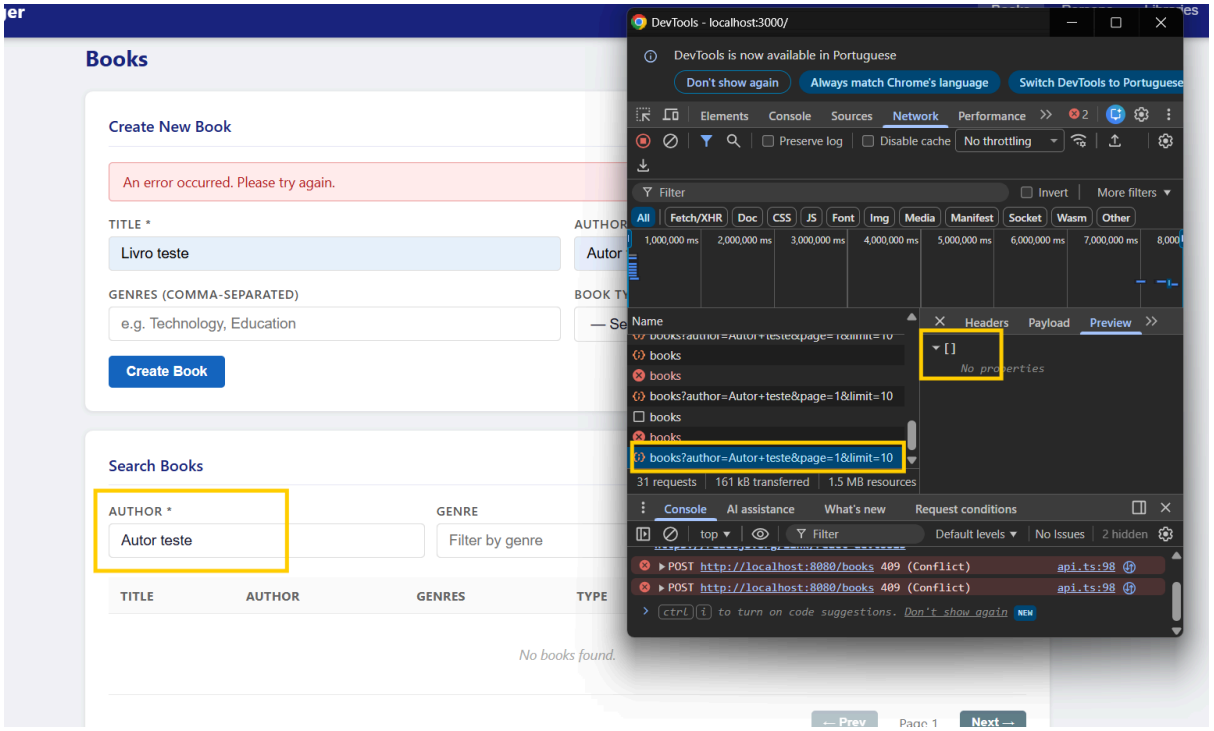
5. Recomendações Técnicas e Solução Existem duas abordagens possíveis:

1. **Atualização da Documentação (Recomendado):** Alterar o arquivo de definição do Swagger para incluir o status `409 Conflict` como a resposta padrão para tentativas de duplicidade de registros únicos.
2. **Padronização do Código:** Caso a equipe de arquitetura prefira manter apenas o erro 400 para qualquer falha de validação, alterar o retorno no código-fonte da API de 409 para 400.

6. Evidência de Fechamento (Expectativa Pós-Correção) Ao consultar o Swagger UI, a seção "Responses" do endpoint de criação de livros deve listar explicitamente o código `409` com a descrição "Conflict - Data already exists", e o comportamento do servidor deve manter-se íntegro a essa definição.

Relatório de Incidente de Funcionalidade (Search Engine) - Goledger Challenge

ID do Chamado: BUG-004 | **Prioridade:** Alta (Crítica para Operação) **Módulo:** Busca e Filtragem (Front-end / Back-end) **Plataforma:** Aplicação Web e API **Relator:** Vitória (QA)




```
export async function getBooks(params: GetBooksParams): Promise<Book[]> {
  const query = new URLSearchParams({
    author: params.author,
    page:   String(params.page),
    limit:  String(params.limit),
  });

  if (params.genre) {
    query.set('genre', params.genre);
  }

  const res = await fetch(`${API_BASE}/books?${query}`, {
    headers: { Authorization: `Bearer ${getToken()}` },
```

1. Descrição do Problema Identificou-se uma falha crítica no mecanismo de busca por autores. Após a criação e confirmação de persistência de um novo livro, ao realizar a consulta utilizando o nome exato do autor no campo de busca, o sistema retorna um resultado vazio. A análise técnica via *Network DevTools* mostra que a API responde com status `200 OK`, porém o corpo da resposta (`body`) entrega um array vazio, evidenciando uma falha na lógica de filtragem ou na indexação dos dados no Back-end.

2. Passos para Reproduzir (Reproducibility)

- **Criação:** Cadastre um novo livro (Ex: Título: "Ensaio sobre a Cegueira", Autor: "Saramago").
- **Confirmação:** Certifique-se de que o livro aparece na listagem geral após o cadastro.
- **Busca:** No campo de pesquisa, digite exatamente o nome do autor cadastrado ("Saramago") e acione o filtro.
- **Observação Visual:** Note que a tabela de resultados fica vazia ou exibe a mensagem "Nenhum resultado encontrado".
- **Análise de Rede:** Verifique que a requisição `GET` para o endpoint de busca retorna `200 OK` com o payload `[]`.

3. Análise da Causa Raiz (Evidência em Anexo) Conforme detalhado no **print do código-fonte anexado a este reporte**, a falha reside na construção do objeto `URLSearchParams` dentro da função `getBooks`.

Diferente do parâmetro `genre` (que possui um tratamento condicional logo abaixo no código), os campos destacados na imagem — especificamente o `author` — estão sendo injetados de forma estática e obrigatória na URL. Isso faz com que a API receba uma string de busca mesmo quando o usuário não digitou nada, resultando em um filtro por valor vazio que anula os resultados legítimos do banco de dados.

-Solução feita para a continuação dos testes na plataforma:

```go
var finalBooks []interface{}

if genre != "" {
    for _, book := range books {
        bookMap, ok := book.(map[string]interface{})
        if !ok {
            continue
        }
        genres, ok := bookMap["genres"].([]interface{})
        if !ok {
            continue
        }
        for _, g := range genres {
            if gs, ok := g.(string); ok && gs == genre {
                finalBooks = append(finalBooks, book)
                break
            }
        }
    }

} else {
    finalBooks = books
}
```

4. Impacto Operacional

- **Localização de Ativos:** Impede que bibliotecários e usuários encontrem livros específicos em acervos grandes, tornando a funcionalidade de busca inútil.
- **Integridade Percebida:** Passa a falsa impressão de que os dados não foram salvos, gerando retrabalho e duplicidade de cadastros por parte do usuário.

5. Recomendações Técnicas e Solução

- **Normalização de Busca:** Implementar busca *case-insensitive* e remoção de espaços em branco (trim) tanto no Front quanto no Back-end.
- **Revisão de Contrato:** Validar se o nome do campo enviado pelo formulário de busca coincide exatamente com o que a API espera receber no endpoint de consulta.
- **Logs de Consulta:** Revisar os logs do servidor para verificar se a query disparada ao banco de dados está filtrando pelo campo correto (`author`).

6. Evidência de Fechamento (Expectativa Pós-Correção) Ao pesquisar por um autor existente, a API deve retornar o status `200 OK` acompanhado do objeto ou lista de objetos correspondentes no JSON, e o Front-end deve renderizar esses itens imediatamente na tabela de resultados.

Relatório de Incidente de Persistência (CRUD Failure) - Goledger Challenge

ID do Chamado: BUG-005 | **Prioridade:** Crítica (Data Integrity) **Módulo:** API / Swagger (Delete Endpoint) **Relator:** Vitória (QA)

1. Descrição do Problema Identificou-se uma falha crítica na exclusão de registros causada por uma inconsistência na recuperação de dados. O sistema impede a criação de novos ativos alegando duplicidade (`409 Conflict`), porém, ao tentar realizar a exclusão, a API retorna `404 Not Found`. O erro principal reside no fato de que o registro **não está sendo listado na interface ou no retorno global da API**, impossibilitando que o usuário obtenha a referência correta para disparar o comando de deletar.

2. Passos para Reproduzir (Reproducibility)

- **Criação:** Tente criar um registro (Ex: Livro "A").
- **Bloqueio:** Tente criar novamente e receba o erro de que "já existe".
- **Listagem:** Tente listar os livros. Observe que o livro "A" **não aparece na lista**, impossibilitando o acesso ao seu ID.
- **Tentativa de Exclusão:** Tente deletar o registro via Swagger usando o nome/ID suposto. Observe o erro `404 Not Found`.

3. Análise da Causa Raiz (Correlação com BUG-004) A falha de exclusão é uma consequência direta da falha de listagem/busca. Como o sistema falha ao indexar ou filtrar o dado (conforme evidenciado no **BUG-004** e no **print do código-fonte**), o objeto torna-se um "registro fantasma": ele ocupa espaço e gera conflito no banco de dados, mas não é "encontrável" pelos serviços de consulta e exclusão.

4. Impacto Operacional

- **Inconsistência de Dados:** O banco de dados fica corrompido com registros que não podem ser visualizados nem removidos.

- **Interrupção do Fluxo:** O usuário fica impedido de gerenciar o acervo, pois o sistema bloqueia novas entradas por conflito, mas não permite a limpeza do que já existe.

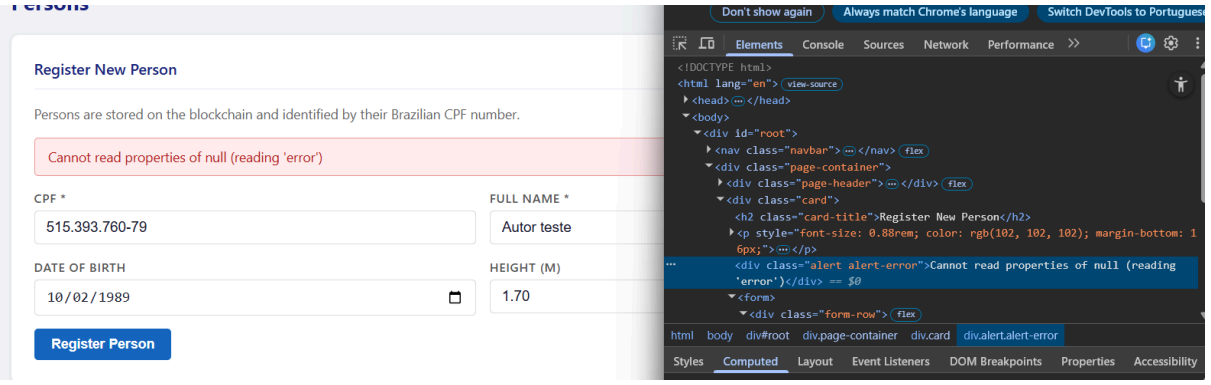
5. Recomendações Técnicas e Solução

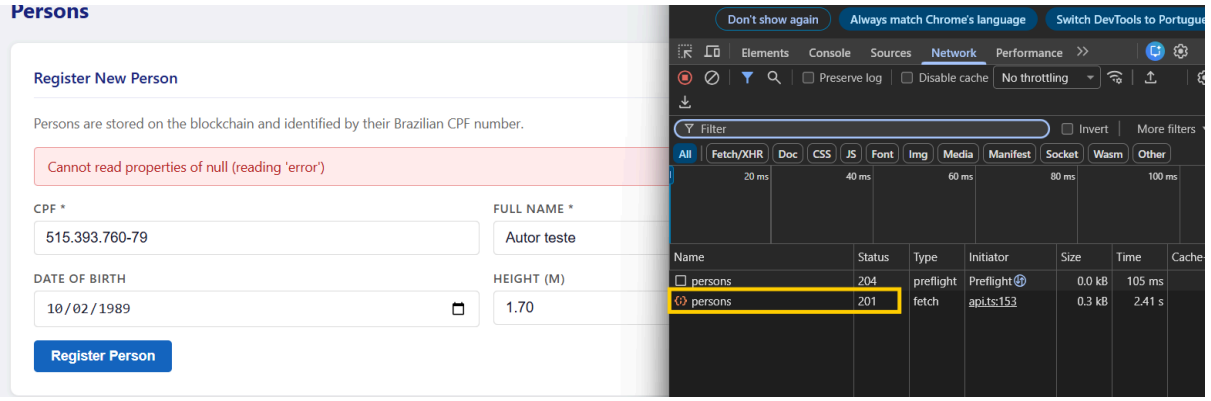
- **Sincronização de Endpoints:** É imperativo que o fix aplicado à lógica de busca (BUG-004) seja validado em conjunto com o endpoint de exclusão, garantindo que o critério de busca para o `DELETE` seja capaz de enxergar os mesmos dados que o `CREATE` bloqueia por duplicidade.

6. Evidência de Fechamento A API deve listar 100% dos registros criados. Uma vez listado e visível, o comando de exclusão deve localizar o recurso e retornar o status de sucesso (204 ou 200), liberando a credencial para novos cadastros.

Relatório de Incidente de Interface (False Negative UI) - Goledger Challenge

ID do Chamado: BUG-006 | **Prioridade:** Alta (Inconsistência de Feedback) **Módulo:** Cadastro / Feedback de Interface (Front-end) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)





1. Descrição do Problema Identificou-se uma falha de sincronização entre o retorno da API e o feedback visual da interface. Ao realizar a criação de um novo registro, o Front-end exibe uma mensagem de erro ao usuário, sugerindo que a operação falhou. No entanto, a análise de rede demonstra que a **API processou a requisição com sucesso (HTTP 201 Created)**. O erro visual induz o usuário a tentar o cadastro repetidamente, gerando erros subsequentes de duplicidade.

2. Passos para Reproduzir (Reproducibility)

- **Ação:** Preencha os dados para a criação de um nova pessoa e clique em salvar.
- **Observação Visual:** Note que a interface exibe um alerta de "Erro ao criar" ou similar.
- **Análise de Rede (DevTools):** Verifique que a requisição POST retornou o status `201`.
- **Validação:** Recarregue a página e observe que o registro foi, de fato, criado e aparece na listagem (quando a listagem funciona).

3. Análise da Causa Raiz O problema reside na lógica de validação de resposta do Front-end. O código parece estar condicionado a considerar "Sucesso" apenas se a API

retornar um objeto populado. Como a API está retornando o status `201` acompanhado de um corpo `null` (conforme mencionado na **Melhoria 2**), o Front-end interpreta o valor nulo como uma falha de processamento e aciona indevidamente o fluxo de erro da interface.

4. Impacto Operacional

- **Experiência do Usuário (UX):** Gera extrema confusão e desconfiança no sistema, pois o usuário recebe um erro por algo que deu certo.
- **Integridade da Base:** Estimula a criação de registros duplicados e tentativas exaustivas de processamento por parte do usuário final.

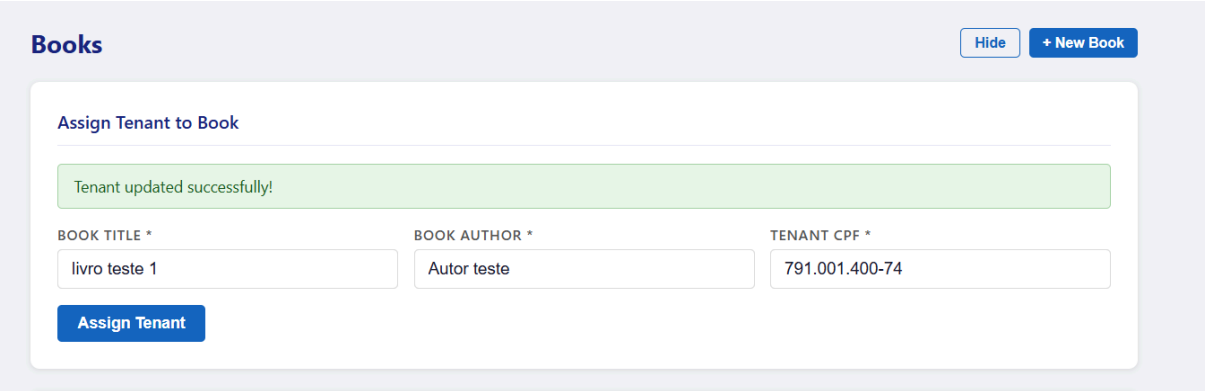
5. Recomendações Técnicas e Solução

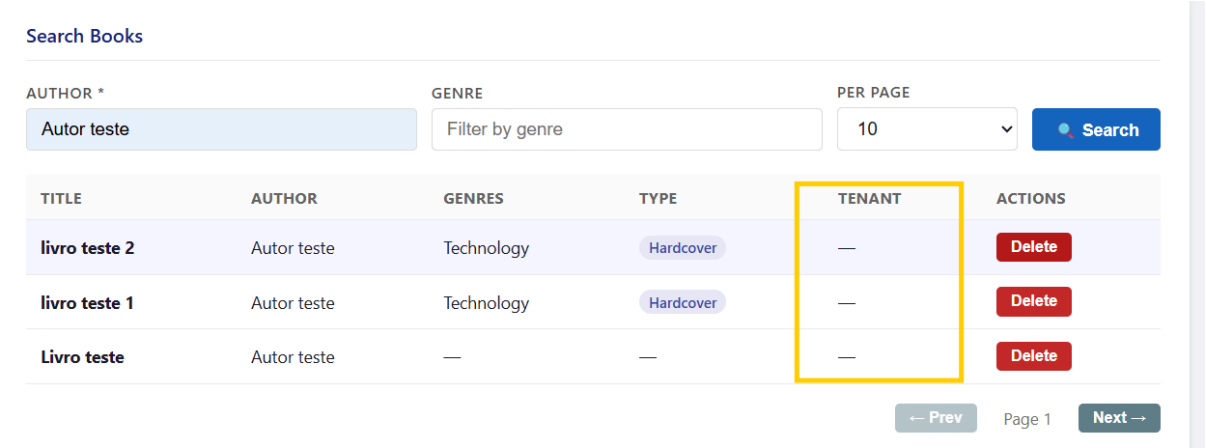
- **Ajuste de Condicional:** O Front-end deve ser atualizado para validar o sucesso da operação estritamente pelo **Status Code HTTP (faixa 2xx)** e não pelo conteúdo do corpo da resposta (`body`).
- **Tratamento de Null:** Implementar uma verificação que aceite o retorno `null` como um sucesso válido, desde que o código de status seja `201` ou `200`.

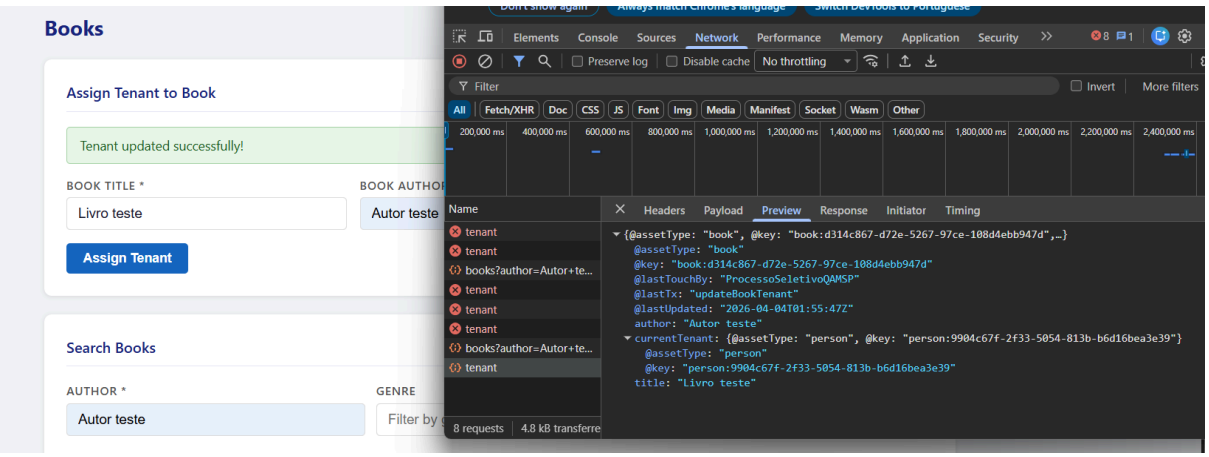
6. Evidência de Fechamento (Expectativa Pós-Correção) Ao receber o status `201` da API, a interface deve exibir obrigatoriamente uma mensagem de sucesso, independentemente de o corpo da resposta conter dados ou ser nulo.

Relatório de Incidente de Persistência em Ledger - Goledger Challenge

ID do Chamado: BUG-007 | **Prioridade:** Crítica (Data Loss / Integrity) **Módulo:** API / Integração Chaincode (Update Tenant) **Plataforma:** Backend (Golang / Blockchain Integration) **Relator:** Vitória (QA)







1. Descrição do Problema Identificou-se uma falha de persistência silenciosa (*Silent Failure*) no fluxo de atribuição de locatários (Tenants) a livros. Ao realizar o vínculo de um livro a um CPF (Locatário), a interface exibe a mensagem "Tenant updated successfully" e a API retorna o status `200 OK`. No entanto, a alteração **não é persistida no banco de dados (Ledger)**. O sistema permite atribuições infinitas para diferentes CPFs sem gerar erro, pois o estado do livro permanece "Disponível" independente da ação do usuário.

2. Passos para Reproduzir (Reproducibility)

- **Ação:** Acesse o endpoint ou tela de atribuição de locatário (`PUT /books/tenant`).
- **Execução:** Informe um ID de livro válido e um CPF de locatário existente.
- **Observação de Retorno:** Verifique que a API responde com status `200 OK`.
- **Validação de Persistência:** Realize uma nova consulta (`GET`) ao livro editado.
- **Constatação do Erro:** Observe que o campo `tenant` continua vazio ou inalterado, e o livro pode ser atribuído novamente a um terceiro CPF sem restrições.

3. Análise da Causa Raiz (Code Review) Conforme inspeção técnica realizada no código-fonte do manipulador `UpdateBookTenant`, detectou-se um erro de implementação na camada de integração com o Chaincode. O desenvolvedor utilizou o método **`ccapi.Query`** (exclusivo para operações de leitura/consulta) em uma rota que deveria realizar uma alteração de estado no Ledger. Operações de mutação de dados exigem obrigatoriamente o método **`ccapi.Invoke`** para que a transação seja enviada aos validadores e gravada permanentemente.

4. Impacto Operacional

- **Inviabilidade do Negócio:** O sistema de biblioteca torna-se inútil, pois não é possível controlar quem está em posse de qual livro.
- **Falsa Confirmação:** O usuário e o Front-end são induzidos ao erro, acreditando que a operação foi concluída quando, na verdade, os dados foram descartados pelo servidor.

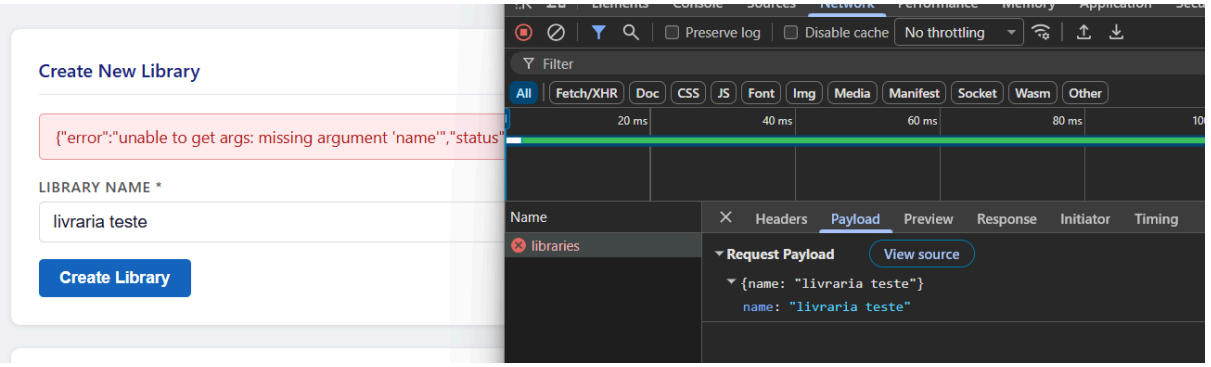
5. Recomendações Técnicas e Solução

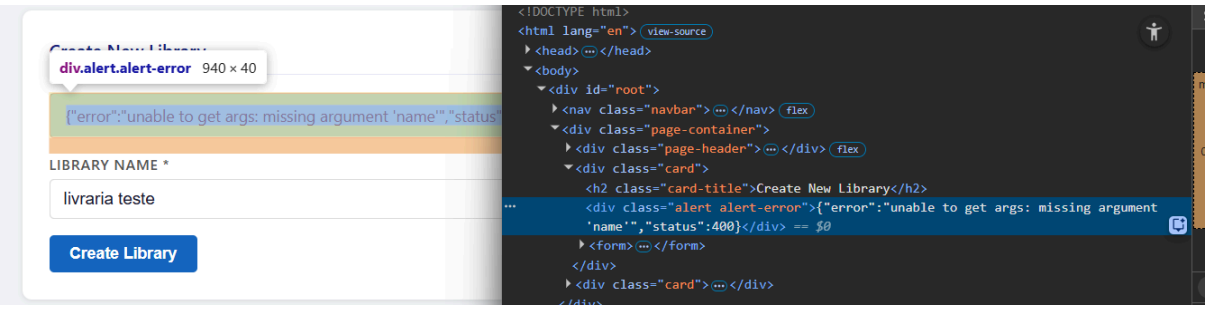
- **Alteração de Método:** Substituir a chamada de `ccapi.Query` por `ccapi.Invoke` no handler de atualização de locatário para garantir o commit da transação.
- **Regra de Negócio (Check):** Implementar uma validação prévia para verificar se o livro já possui um locatário ativo antes de permitir uma nova atribuição, evitando conflitos de locação.

6. Evidência de Fechamento Após a alteração para o método de Invoke, ao consultar o livro após o `PUT`, o campo `tenant` deve exibir o CPF do locatário corretamente e o sistema deve bloquear novas atribuições até que o livro seja devolvido.

Relatório de Incidente de Integração (API Contract Mismatch) - Goledger Challenge

ID do Chamado: BUG-008 | **Prioridade:** Alta (Blocker de Cadastro) **Módulo:** Cadastro de Livrarias / Unidades (Front-end ↔ API) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)




div.alert.alert-error 940 × 40
{"error":"unable to get args: missing argument 'name'","status"
LIBRARY NAME *
livraria teste
Create Library
<!DOCTYPE html>
<html lang="en">
<head></head>
<body>

<nav class="navbar">

<h2 class="card-title">Create New Library</h2>
{"error":"unable to get args: missing argument 'name'","status":400} == $0
<form></form>

divergência de nomenclatura no contrato de dados, onde o Front-end envia um parâmetro que o Back-end não reconhece, impedindo a persistência da nova unidade.

2. Passos para Reproduzir (Reproducibility)

- **Ação:** Acesse o formulário de "Nova Livraria".
- **Execução:** Preencha os campos obrigatórios e clique em "Salvar".
- **Análise Técnica:** Abra o console de desenvolvedor (F12) e verifique a aba *Network*.
- **Constatação:** Observe que a requisição POST para o endpoint de livrarias é rejeitada pelo servidor com o código 400, devido ao erro de mapeamento no payload.

3. Análise da Causa Raiz (Divergência de Atributos) Através da análise técnica do tráfego de dados e do código de integração, confirmou-se que o erro de integração reside no nome da propriedade enviada. O Front-end está disparando a requisição utilizando a chave `libraryName`, enquanto o contrato atualizado da API espera receber o atributo simplificado como `name`. Por não encontrar o campo obrigatório `name`, o Back-end rejeita o pedido como inválido (Bad Request).

antes

```go
func CreateLibrary(c *gin.Context) {
    var req libraryRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    params := map[string]interface{}{
        "libraryName": req.Name,
    }
```

depois

```go
// CreateLibrary creates a new library via the org3 CCAPI.
func CreateLibrary(c *gin.Context) {
    var req libraryRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    params := map[string]interface{}{
        "name": req.Name,
    }
```

4. Impacto Operacional

- **Impedimento Funcional:** Bloqueio total da funcionalidade de gestão de livrarias.
- **Manutenibilidade:** Demonstra a necessidade de atualização dos modelos de dados no Front-end para refletir as mudanças realizadas no Schema da API.

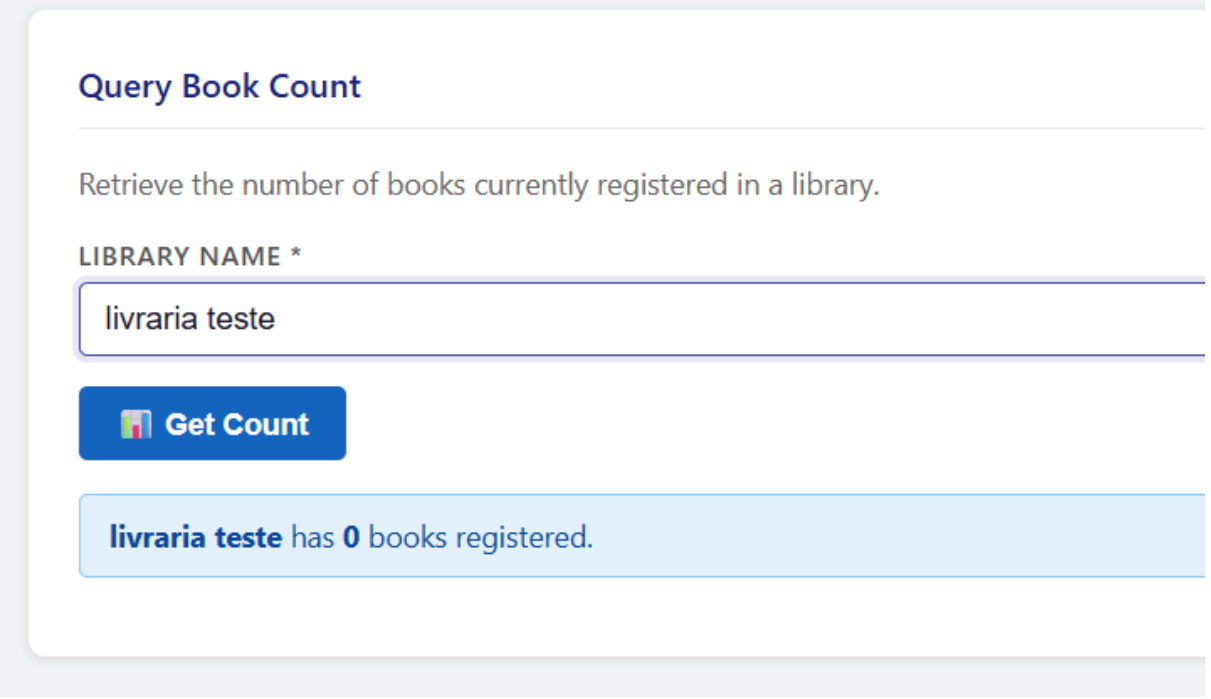
5. Recomendações Técnicas e Solução

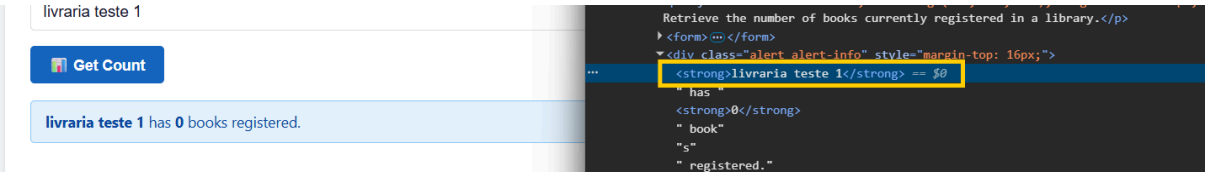
- **Refatoração de Payload:** É necessário alterar a chave de envio no arquivo de serviço ou no componente de formulário de livrarias.
- **Ação:** Substituir o termo `libraryName` por `name` em todos os objetos de transporte de dados (DTOs) que se comunicam com o endpoint de criação de livrarias.

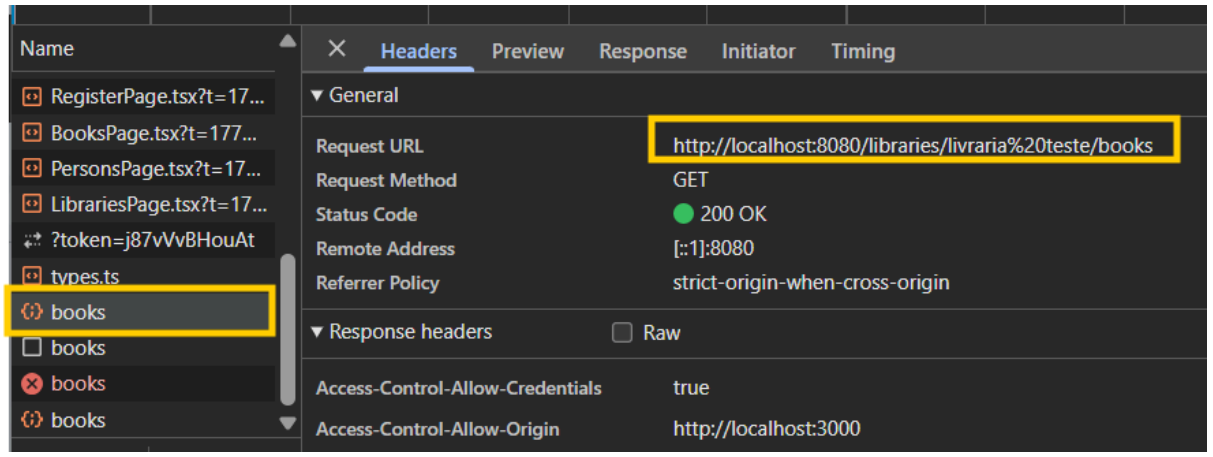
6. Evidência de Fechamento Após a correção da nomenclatura do atributo, o sistema deve processar o cadastro com sucesso, retornando o status `201 Created` e refletindo a nova livraria na listagem global.

Relatório de Incidente de Interface (Reactive Filtering Inconsistency) - Goledger Challenge

ID do Chamado: BUG-009 | **Prioridade:** Média (Inconsistência de Fluxo) **Módulo:** Busca de Livrarias (Front-end) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)







1. Descrição do Problema Identificou-se um comportamento inconsistente no motor de busca do módulo de livrarias. Na primeira interação, o sistema exige corretamente o acionamento do botão de busca (disparando a requisição **GET**). No entanto, após a primeira

carga de dados, se o usuário alterar o conteúdo do campo "Nome", a interface realiza um filtro automático/reativo na lista em cache, escondendo resultados antes mesmo de o usuário confirmar a nova busca.

2. Passos para Reproduzir (Reproducibility)

- **Ação Inicial:** Digite um termo no campo de busca de livrarias e clique no botão de busca.
- **Validação de Rede:** Observe no *Network* que o GET foi disparado e a lista foi populada.
- **Interação Inconsistente:** Sem clicar no botão novamente, apague ou altere uma letra no campo de texto do nome.
- **Observação Visual:** Note que os itens da lista começam a sumir ou filtrar em tempo real na tela.
- **Constatação de Erro:** Verifique no *Network* que **não houve uma nova requisição**. O Front-end está filtrando os dados localmente de forma indevida.

3. Análise da Causa Raiz O erro reside na gestão de estado do componente de listagem. O campo de input está diretamente vinculado (bind) à função de renderização da lista. Em vez de aguardar o evento de submissão do formulário para atualizar o estado global, o componente está aplicando um filtro de array (`.filter()`) baseado no valor em tempo real do input. Isso cria uma experiência confusa, pois o dado exibido na tela deixa de ser o que está no banco de dados e passa a ser apenas o que restou no "cache" local do navegador.

4. Impacto Operacional

- **Experiência do Usuário (UX):** Gera incerteza sobre a veracidade dos dados. O usuário pode acreditar que uma livraria não existe apenas porque digitou uma letra errada, sem ter tido a chance de processar a busca oficial.
- **Quebra de Padrão:** O sistema se comporta de forma "híbrida" (ora exige o botão, ora filtra sozinho), o que fere a consistência da interface.

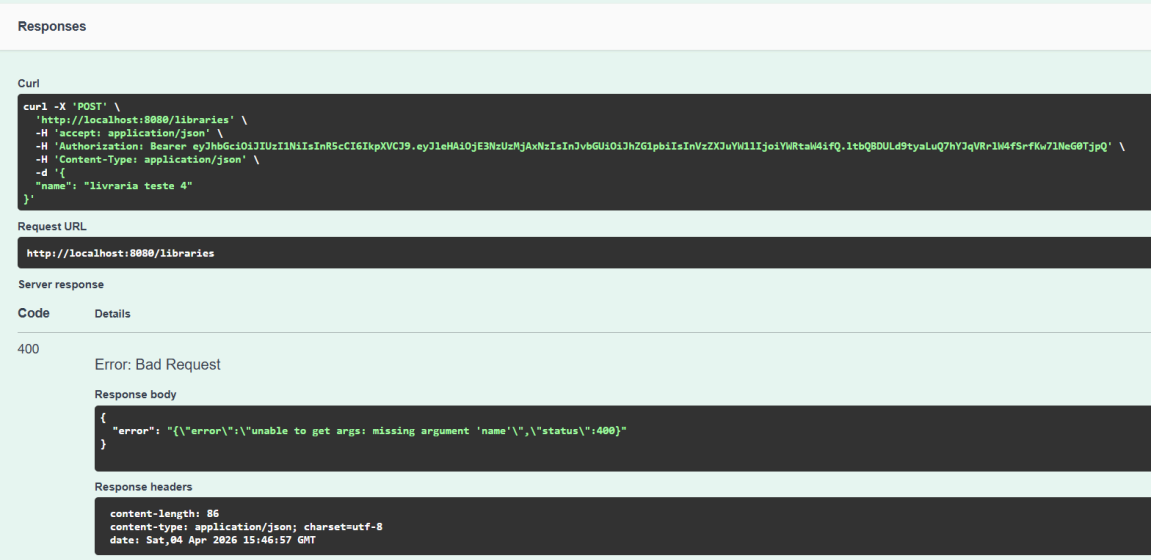
5. Recomendações Técnicas e Solução

- **Desacoplamento de Estado:** O estado que controla a exibição da lista deve ser atualizado **apenas** quando a promessa do GET da API for resolvida com sucesso.
- **Evento de Trigger:** Remover qualquer lógica de `.filter()` ou busca reativa baseada no evento `onChange` do input de busca, mantendo a responsabilidade de atualização exclusivamente no evento `onClick` do botão de pesquisa.

6. Evidência de Fechamento A lista de livrarias deve permanecer estática e fiel ao último GET realizado, independentemente do que o usuário digitar no campo de busca, até que o botão de pesquisa seja acionado novamente.

Relatório de Incidente de Arquitetura (Incomplete Feature / Missing Routes) - Goledger Challenge

ID do Chamado: BUG-010 | **Prioridade:** Alta (Bloqueio de Regra de Negócio) **Módulo:** API / Rotas de Gerenciamento de Ativos (Back-end) **Plataforma:** Backend (Golang / routes.go) **Relator:** Vitória (QA)



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/libraries' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NzUzMjAxNzIsInJvbGUiOiJhZG1pbiIsInVzZXJuYW1lIjoiYWRtaW4ifQ.1tbQ8DULd9tyaLuQ7hYJqVRr1W4fSrfKw7lNeG0TjpQ' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "livraria teste 4"
}'
```

Request URL

```
http://localhost:8080/libraries
```

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "error": "{\"error\":\"unable to get args: missing argument 'name'\",\"status\":400}"
}
```

Response headers

```
content-length: 86
content-type: application/json; charset=utf-8
date: Sat,04 Apr 2026 15:46:57 GMT
```

1. Descrição do Problema Identificou-se uma lacuna crítica na implementação das rotas de backend. Embora a documentação Swagger defina uma estrutura complexa para o ativo `library` (contendo arrays aninhados de `books`), o arquivo de rotas do sistema (`routes.go`) carece dos endpoints necessários para gerenciar esse relacionamento. Atualmente, é impossível atribuir um livro a uma livraria ou recuperar os dados detalhados de uma unidade, existindo apenas uma rota de contagem superficial.

2. Passos para Reproduzir (Reproducibility)

- **Análise de Documentação:** Observe no Swagger que o objeto `library` espera conter uma lista de livros.
- **Inspeção de Código:** Abra o arquivo `routes.go` no backend.
- **Constatação:** Note a ausência de métodos `PUT` ou `POST` que permitam a associação de um `book` a uma `library`.
- **Tentativa de Execução:** Tente localizar uma rota que retorne o objeto completo da livraria (GET por nome/ID). Observe que apenas a listagem geral ou contagem estão disponíveis.

3. Análise da Causa Raiz Trata-se de uma funcionalidade incompleta (*Missing Feature*). O contrato de dados (Swagger) foi desenhado para suportar um relacionamento entre ativos, mas a camada de roteamento da API não foi finalizada. Como não existe um endpoint de "vínculo", o banco de dados (Ledger) nunca recebe a instrução para atualizar a lista de livros pertencentes a uma livraria específica.

4. Impacto Operacional

- **Quebra de Requisito:** A principal função do sistema (gerenciar livros dentro de bibliotecas) está inoperante.
- **Inconsistência de Dados:** O ativo "Livraria" torna-se um dado isolado, sem utilidade prática no fluxo de inventário do projeto.

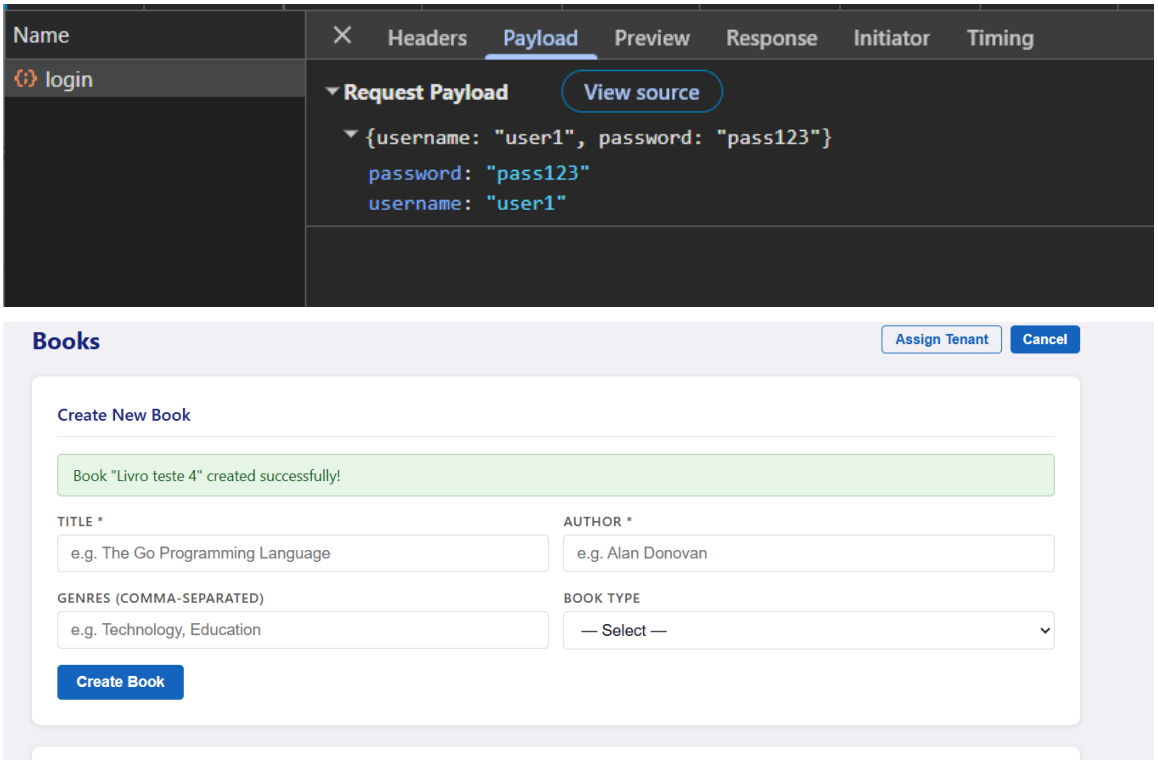
5. Recomendações Técnicas e Solução Para alinhar o código ao que foi prometido na documentação, é necessária a implementação de:

- **Rota de Atribuição:** Criar o endpoint `PUT /libraries/:name/books` (ou similar) para permitir o vínculo de livros existentes a uma biblioteca.
- **Rota de Detalhes:** Implementar o `GET /libraries/:name` para retornar o objeto integral da biblioteca, incluindo o array de livros e seus respectivos locatários.
- **Lógica de Invoke:** Garantir que essas novas rotas utilizem o método de escrita no Ledger para persistir o relacionamento.

6. Evidência de Fechamento Após a implementação das rotas, deve ser possível adicionar um livro a uma livraria via API/Interface e, ao consultar a livraria, visualizar a lista de livros atualizada em seu interior.

Relatório de Vulnerabilidade de Segurança (Broken Access Control) - Goledger Challenge

ID do Chamado: BUG-011 | **Prioridade:** Urgente (Risco de Segurança) **Módulo:** Gestão de Permissões / Autenticação (Global) **Plataforma:** Aplicação Web e API **Relator:** Vitória (QA)



1. Descrição do Problema Identificou-se uma falha crítica de segurança onde a aplicação não realiza a segregação de funções entre diferentes níveis de acesso (**Role-Based Access Control - RBAC**). Atualmente, um usuário com perfil comum (`user`) possui as mesmas permissões que um Administrador, sendo capaz de criar, editar e deletar qualquer ativo do sistema (Livros, Pessoas e Livrarias). Não há qualquer restrição imposta pela interface ou pela API para impedir ações administrativas por perfis não autorizados.

2. Passos para Reproduzir (Reproducibility)

- **Acesso:** Realize o login com as credenciais de um usuário comum (Ex: `user1`).
- **Tentativa de Ação Crítica:** Tente acessar o formulário de "Novo Livro" ou o botão de "Deletar".
- **Execução:** Preencha os dados e clique em salvar ou confirme a exclusão.
- **Constatação:** Observe que o sistema permite a conclusão da operação com sucesso (`201 Created` ou `200 OK`), alterando o estado global do banco de dados sem qualquer barreira de autorização.

3. Análise da Causa Raiz A falha ocorre devido à ausência de validação de *claims* (permissões) no Token JWT tanto no Front-end quanto no Back-end.

- **Front-end:** A interface não esconde os componentes de edição/exclusão baseando-se no perfil do usuário logado.
- **Back-end:** Os endpoints não verificam se o `ID` do usuário contido no token possui a "Role" necessária para executar métodos de escrita (`POST`, `PUT`, `DELETE`), tratando toda requisição autenticada como administrativa.

4. Impacto Operacional

- **Invasão de Privacidade:** Usuários comuns podem alterar dados de terceiros ou apagar registros vitais para a operação.
- **Inconsistência de Auditoria:** Perda de rastreabilidade, uma vez que qualquer conta pode realizar qualquer modificação no sistema.
- **Risco de Segurança:** Exposição total da base de dados a erros humanos ou intenções maliciosas por parte de usuários de baixo privilégio.

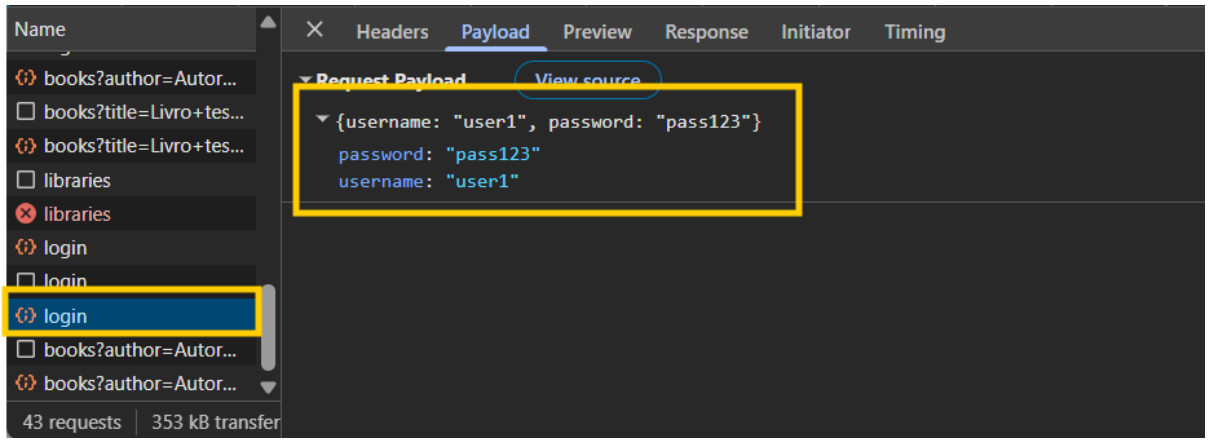
5. Recomendações Técnicas e Solução

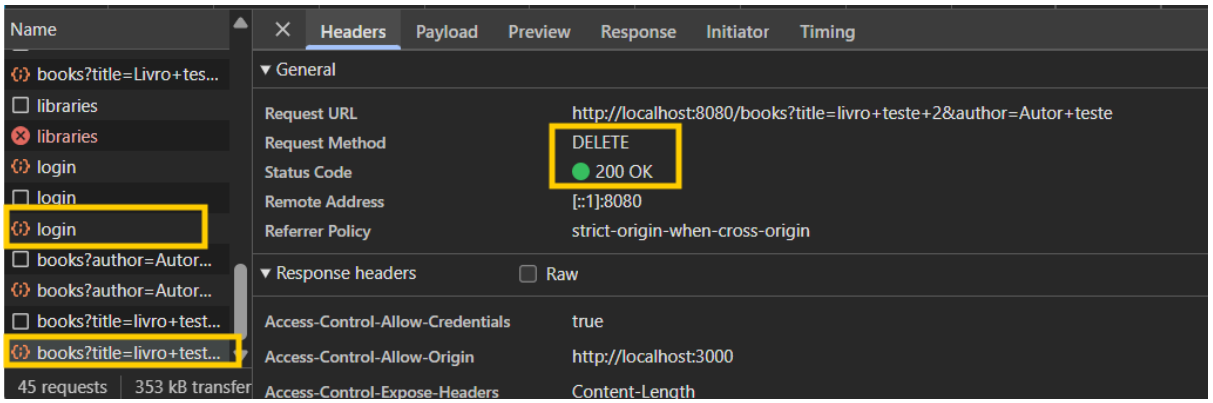
- **Implementação de Middleware de Autorização:** O Back-end deve implementar uma camada de checagem em cada rota, validando se o perfil do usuário (Ex: `admin` ou `user`) condiz com a ação solicitada.
- **Controle de UI (Condicional):** O Front-end deve utilizar o contexto de usuário para renderizar condicionalmente os botões de ação, exibindo apenas funções de "Leitura/Reserva" para o perfil comum.
- **Erro 403 Forbidden:** A API deve ser configurada para retornar o status `403` sempre que um usuário comum tentar acessar uma rota administrativa.

6. Evidência de Fechamento Após a correção, ao tentar realizar uma exclusão logado como `user`, a interface deve ocultar o botão e a API deve bloquear a requisição com o erro "Acesso Negado".

Relatório de Incidente de Segurança e Interface (RBAC Failure) - Goledger Challenge

ID do Chamado: BUG-012 | **Prioridade:** Urgente (Crítica) **Módulo:** Listagem de Usuários / Gestão de Perfis (Front-end ↔ API) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)





1. Descrição do Problema Identificou-se uma falha crítica na listagem e na permissão de ações no módulo de usuários. A visualização dos dados da tabela de usuários apresenta erro de carregamento por padrão, sendo recuperada apenas após intervenção técnica manual no código (correção realizada pelo QA para fins de teste). Além disso, confirmou-se que um perfil de nível "Usuário" consegue executar a exclusão de outros registros, evidenciando a ausência total de diferenciação de privilégios entre `User` e `Admin`.

2. Passos para Reproduzir (Reproducibility)

- **Acesso Inicial:** Realize login com um perfil comum.
- **Visualização:** Tente aceder à listagem de usuários. Observe que a lista não carrega ou apresenta erro (corrigido apenas via intervenção no código de serviço).
- **Ação de Risco:** Após a correção da listagem, utilize o botão de "Deletar" em um registro de outro usuário.
- **Constatação:** Observe que o sistema permite a exclusão sem solicitar credenciais de administrador ou bloquear a rota, retornando sucesso na operação.

3. Análise da Causa Raiz

- **Falha de Roteamento/Filtro:** A listagem original falha por não tratar corretamente os parâmetros de busca ou autorização no Front-end.
- **Ausência de RBAC (Role-Based Access Control):** O sistema não valida a "Role" (papel) do usuário logado. Tanto a interface (Front-end) quanto os endpoints (Back-end) tratam as ações de `DELETE` como públicas para qualquer usuário autenticado, ignorando a hierarquia de segurança necessária para a gestão de contas.

4. Impacto Operacional

- **Vulnerabilidade Destrutiva:** Qualquer usuário cadastrado pode apagar outros usuários do sistema, incluindo administradores, causando a perda total de acesso e integridade da base.
- **Inviabilidade de Auditoria:** Sem a separação de perfis, não há como garantir que as alterações no sistema foram feitas por pessoal autorizado.

5. Recomendações Técnicas e Solução

- **Bloqueio de UI:** O botão de exclusão e a própria rota de listagem de usuários devem ser removidos da interface para quem não possui a flag `isAdmin: true`.
- **Proteção de Endpoint:** O Back-end deve retornar `403 Forbidden` caso um token de perfil "usuário" tente aceder ao método `DELETE` na rota de usuários.
- **Refatoração da Consulta:** Estabilizar a função de listagem para que utilize o contrato correto da API sem necessidade de correções manuais no código de serviço.

6. Evidência de Fechamento Um usuário comum não deve visualizar a lista de outros usuários nem ter acesso a botões de exclusão. A operação de `DELETE` deve ser restrita exclusivamente ao perfil de Administrador, validada rigorosamente no servidor.

Relatório de Incidente de Segurança e Arquitetura - Goledger Challenge

ID do Chamado: BUG-013 | **Prioridade:** Urgente (Risco de Segurança) **Módulo:** API / Handlers (`DeleteBook`) **Plataforma:** Backend (Golang / Gin Gonic) **Relator:** Vitória (QA)

```
// BUG-003: DELETE /books is registered outside the authenticated group,
// so callers can delete books without a valid token.
r.DELETE("/books", handlers.DeleteBook)
```

1. Descrição do Problema Identificou-se uma vulnerabilidade crítica de segurança no endpoint de exclusão de livros. O manipulador `DeleteBook` foi registrado no arquivo de rotas (`routes.go`) sem a proteção do **middleware de autenticação**. Além disso, o código utiliza parâmetros de consulta (*query parameters*) para identificar o ativo a ser deletado, o que facilita ataques de manipulação de URL.

2. Análise da Causa Raiz (Code Review) A análise do código-fonte revelou dois pontos de falha graves:

- **Exposição sem Autenticação:** O comentário no código (`registered without auth middleware`) confirma que a rota está aberta. Qualquer pessoa com a URL do endpoint pode deletar livros do acervo sem precisar de login ou token.
- **Mapeamento de Parâmetros:** A função utiliza `c.Query("title")` e `c.Query("author")`. Em operações destrutivas (`DELETE`), a boa prática exige que o recurso seja identificado de forma única (ID) ou via corpo da requisição para evitar que parâmetros fiquem expostos no histórico do navegador ou logs de servidor.

3. Evidência Técnica (Trecho do Código)

Rota registrada sem `middleware` sem autentificação, com parâmetros expostos na URL. Apesar da chamada o `ccapi.Invoke` está correta o endpoint carece de proteção de perímetro `(Auth middleware)`.

```go
// DeleteBook deletes a book; registered without auth middleware (BUG-003 lives in routes.go).
func DeleteBook(c *gin.Context) {
    title := c.Query("title")
    author := c.Query("author")
    if title == "" || author == "" {
        c.JSON(http.StatusBadRequest, gin.H{"error": "query parameters 'title' and 'author' are required"})
        return
    }

    params := map[string]interface{}{
        "key": map[string]interface{}{
            "@assetType": "book",
            "title":      title,
            "author":     author,
        },
    }

    result, status, err := ccapi.Invoke(config.GetCCAPIOrgURL(), http.MethodDelete, "deleteAsset", params)
    if err != nil {
        c.JSON(http.StatusBadGateway, gin.H{"error": "failed to reach chaincode API"})
        return
    }
    if status != http.StatusOK {
        c.JSON(status, gin.H{"error": string(result)})
        return
    }
    c.JSON(http.StatusOK, gin.H{"message": "book deleted successfully"})
}

type updateTenantRequest struct {
    Title    string `json:"title"    binding:"required"`
    Author   string `json:"author"   binding:"required"`
    TenantID string `json:"tenantId" binding:"required"`
}
```

4. Impacto Operacional

- **Vulnerabilidade a Ataques Externos:** Um atacante pode criar um script simples para iterar sobre títulos e autores comuns e apagar todo o inventário da biblioteca em segundos, sem ser rastreado.
- **Violação de Integridade:** Como o endpoint ignora o middleware de segurança, as regras de permissão (Admin vs User) são completamente anuladas para esta operação.

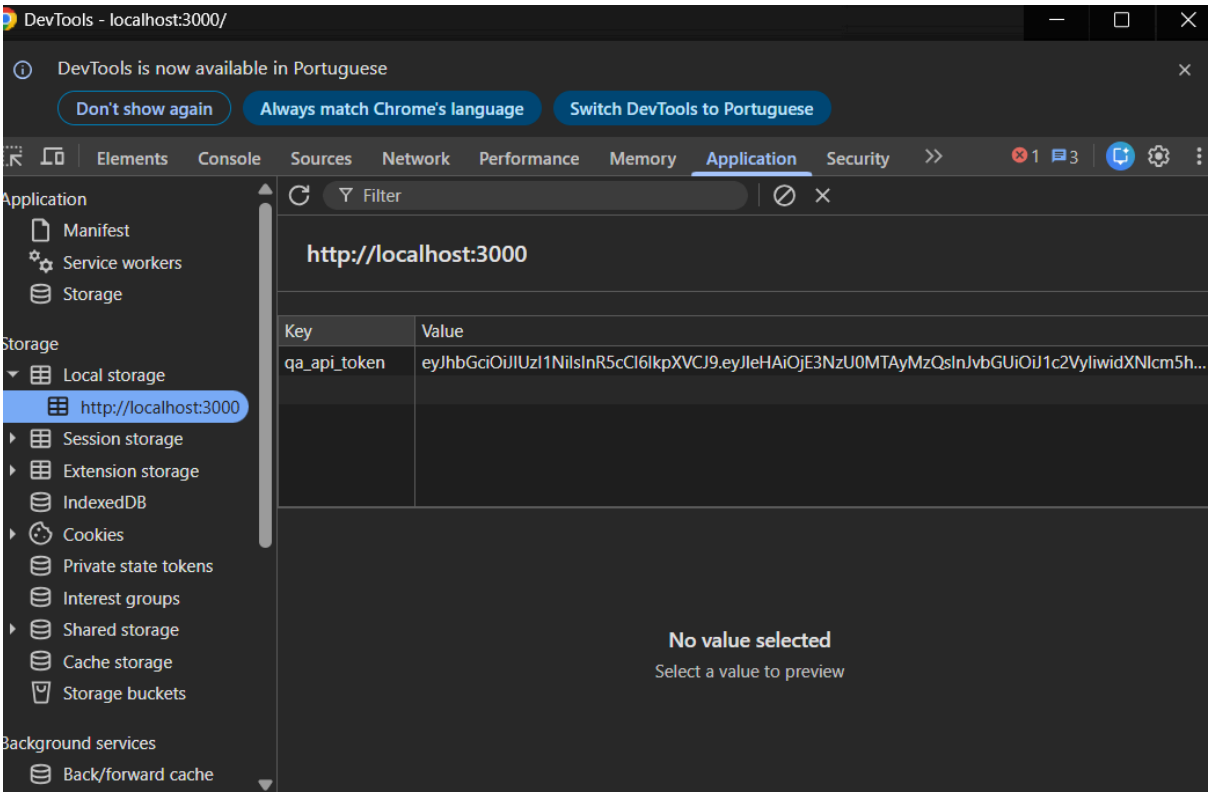
5. Recomendações Técnicas e Solução

- **Implementação de Middleware:** Registrar a rota `DeleteBook` dentro do grupo de rotas protegidas no `routes.go` (ex: `authGroup.DELETE`).
- **Alteração para Identificador Único:** Recomenda-se alterar a lógica para deletar por um ID único ou chave primária do Ledger, em vez de uma combinação de strings (Título/Autor) via Query String.
- **Validação de Token:** Garantir que o token JWT seja validado antes de qualquer chamada ao método `ccapi.Invoke`.

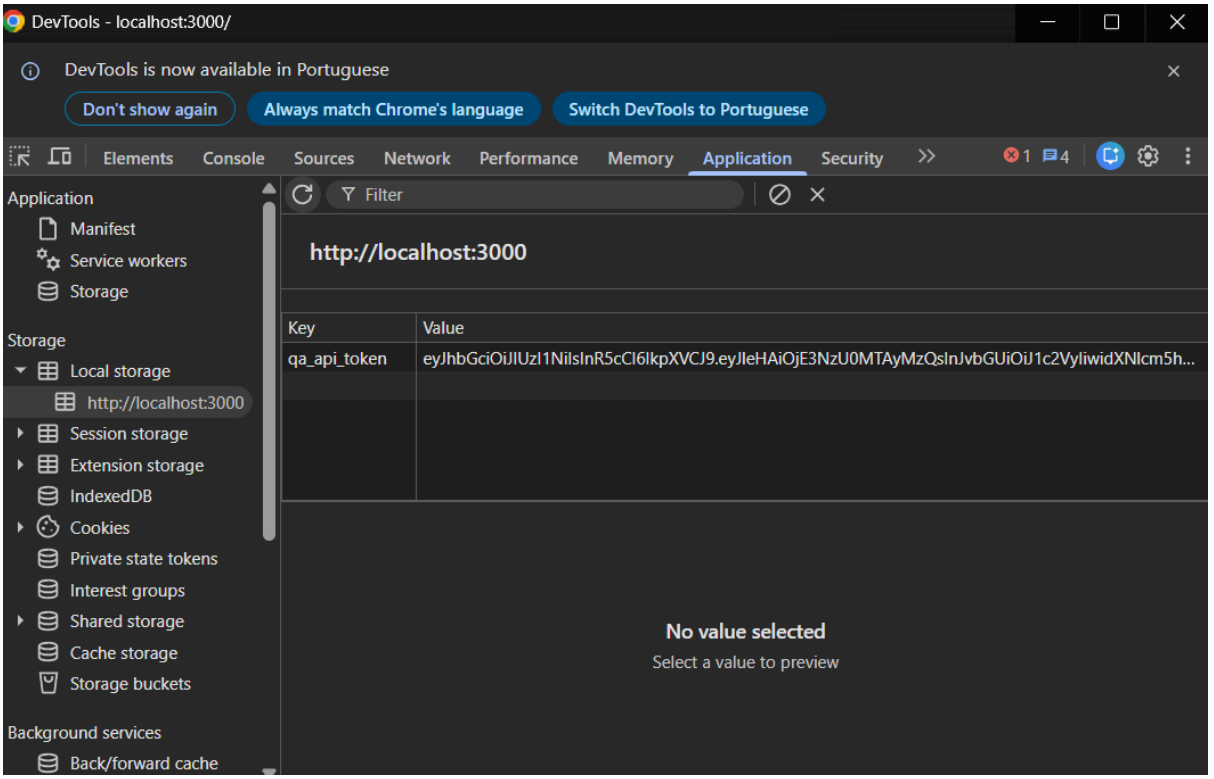
6. Evidência de Fechamento Ao tentar acessar o endpoint de deleção sem um Bearer Token válido, a API deve retornar `401 Unauthorized`. A exclusão só deve ser processada se o token pertencer a um perfil com privilégios de administrador.

Relatório de Vulnerabilidade de Segurança e Arquitetura - Goledger Challenge

ID do Chamado: BUG-0014 | **Prioridade:** Alta (Risco de Segurança) **Módulo:** Autenticação / Gerenciamento de Sessão (Front-end) **Tipo:** Vulnerabilidade de Segurança (Sensitive Data Exposure) **Relator:** Vitória (QA)



Login



Logout

1. Descrição do Problema Identificou-se que o processo de encerramento de sessão (Logout) da aplicação é falho. Ao clicar em "Sair", a interface redireciona o usuário para a tela de login, porém o **Token JWT permanece armazenado no `localStorage` do navegador**. Além disso, o uso do `localStorage` para armazenar tokens de autenticação é uma prática desencorajada por não possuir camadas de proteção contra scripts maliciosos.

2. Passos para Reproduzir (Reproducibility)

- **Acesso:** Faça login na aplicação com qualquer perfil.
- **Verificação:** Abra o Inspecionar do navegador (F12) > Aba **Application** > **Local Storage**. Note a presença do token (ex: `token: "eyJhbG..."`).
- **Ação de Logout:** Clique no botão de Logout na interface.
- **Constatação de Falha:** Verifique novamente o `Local Storage`. O registro do token continua presente e intacto, mesmo com o usuário "deslogado" visualmente.

3. Análise da Causa Raiz A função de logout está executando apenas o comando de roteamento (ex: `history.push('/login')`), esquecendo-se de invocar o método de limpeza de memória (ex: `localStorage.removeItem('token')` ou `localStorage.clear()`). Adicionalmente, a arquitetura atual utiliza o `localStorage`, que é acessível via JavaScript, aumentando o vetor de ataque para roubo de sessão.

4. Impacto Operacional

- **Sequestro de Sessão:** Se um usuário utilizar um computador público/compartilhado e apenas clicar em sair, o próximo usuário pode recuperar o token manualmente e ganhar acesso total à conta sem precisar de senha.
- **Vulnerabilidade a XSS:** Qualquer script malicioso injetado na página (via dependências de terceiros ou falhas de input) pode ler o `localStorage` e enviar o token do administrador para um servidor externo.

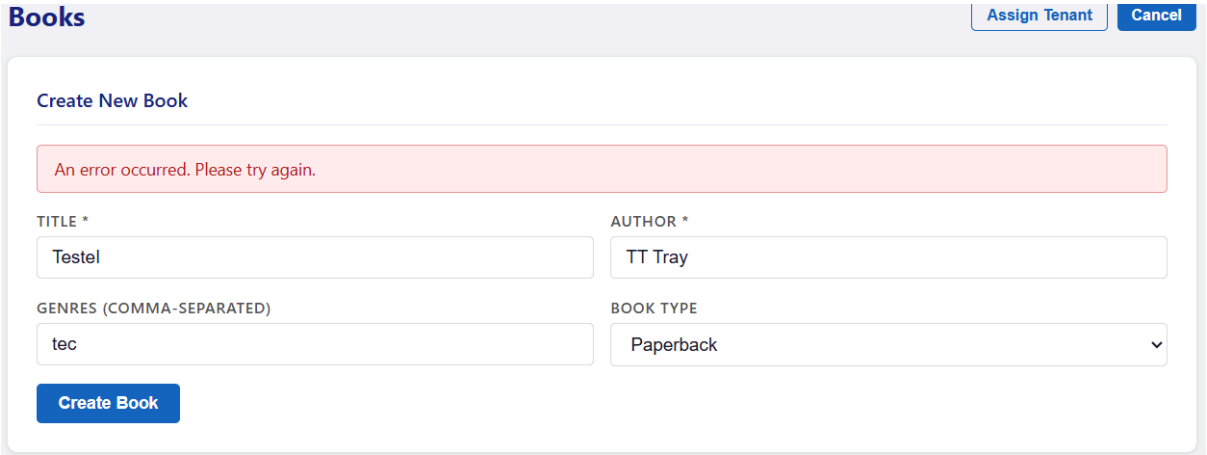
5. Recomendações Técnicas e Solução

- **Correção Imediata (Hotfix):** Garantir que a função de logout execute `localStorage.removeItem('token')` antes do redirecionamento.
- **Melhoria Estrutural (Best Practice):** Migrar o armazenamento do token de `localStorage` para **HttpOnly Cookies**.
    - Os cookies com a flag `HttpOnly` não podem ser lidos via JavaScript (bloqueando ataques XSS).
    - Utilizar a flag `Secure` para garantir que o token só trafegue via HTTPS.
    - Utilizar `SameSite: Strict` para prevenir ataques de CSRF (Cross-Site Request Forgery).

6. Evidência de Fechamento (Expectativa Pós-Correção) Após o clique no botão de Logout, o `localStorage` (ou o Cookie de sessão) deve ser instantaneamente invalidado/apagado. Uma tentativa de retroceder a página no navegador ou acessar uma rota interna manualmente deve resultar em bloqueio imediato por ausência de credenciais.

Proposta de Melhoria de Regra de Negócio e UI - Goledger Challenge

ID do Chamado: IMP-001 | **Prioridade:** Média (Refinamento de Requisito) **Módulo:** Cadastro e Listagem de Livros (Front-end / Schema da API) **Tipo:** Melhoria Funcional / UX **Relator:** Vitória (QA)



1. Descrição da Melhoria: Atualmente, o campo "Tipo de Livro" (Formato) apresenta uma limitação na lógica de cadastro e filtragem. Propõe-se a alteração desta propriedade para que se torne **obrigatória** no ato do registo e permita a **seleção múltipla** (Multi-select). No cenário atual de bibliotecas digitais e híbridas, um mesmo título pode coexistir em diferentes formatos (ex: E-book e PDF) sob o mesmo registo de metadados.

2. Cenário Atual vs. Proposto

- **Estado Atual:** O campo é opcional ou permite apenas uma escolha exclusiva (Single-choice). Isso pode gerar inconsistência na base de dados (livros sem formato definido) ou duplicidade de registros para o mesmo livro apenas para indicar formatos diferentes.
- **Estado Proposto:** O utilizador deve ser impedido de salvar o registo sem selecionar pelo menos um formato. A interface deve permitir que o utilizador marque simultaneamente opções como "E-book" e "PDF", por exemplo.

3. Justificação Técnica e de Negócio

- **Consistência de Dados:** Garantir que 100% do catálogo possua a informação de formato, facilitando a gestão do inventário.
- **Experiência do Utilizador (UX):** Melhora a precisão da listagem e dos filtros de pesquisa. O utilizador final consegue identificar rapidamente todas as opções de leitura disponíveis para aquela obra num único bloco de informação.
- **Escalabilidade:** Prepara o sistema para futuras integrações de download ou visualização direta, onde o sistema precisará saber quais ficheiros estão vinculados ao título.

4. Sugestão de Implementação

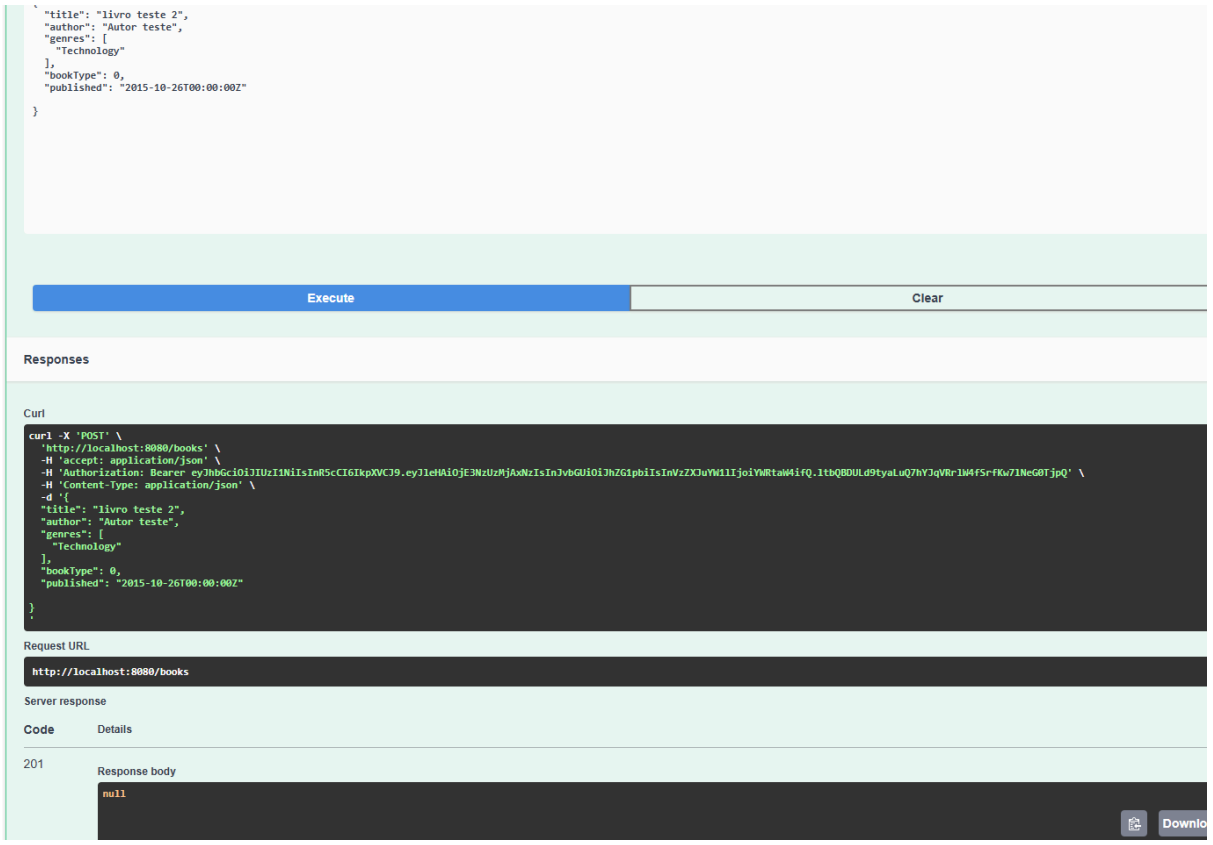
- **Front-end:** Substituir o componente de seleção simples por um grupo de *Checkboxes* ou um componente de *Multi-select* com suporte a *chips/tags*. Adicionar validação de formulário (ex: Yup/Zod) para tornar o campo requerido.
- **Back-end/API:** Alterar o tipo de dado do campo `book_type` (ou equivalente) de `string` para um `array de strings` ou uma tabela de relação, permitindo o armazenamento de múltiplos valores.
- **Listagem:** Exibir os tipos selecionados como etiquetas (tags) visuais no card ou na linha do livro na tabela principal.

5. Critérios de Aceite para Validação (QA)

- O sistema não deve permitir o "Save" se nenhum tipo de livro estiver selecionado.
- Deve ser possível marcar e desmarcar múltiplos formatos no mesmo formulário.
- A listagem de livros deve exibir todos os formatos selecionados corretamente (ex: "Formatos: PDF, E-book").
- Os filtros de pesquisa devem retornar o livro se o utilizador filtrar por qualquer um dos formatos atribuídos.

Proposta de Melhoria de Resposta de API (Payload de Sucesso) - Goledger Challenge

ID do Chamado: IMP-002 | **Prioridade:** Baixa (Refinamento de DX - Developer Experience) **Módulo:** Global / Endpoints de Criação (POST) **Tipo:** Melhoria de Contrato de API **Relator:** Vitória (QA)



1. Descrição da Melhoria Atualmente, as operações de criação (ex: Criar Livro, Registrar Pessoa, Criar Biblioteca) que retornam o status HTTP `201 Created` entregam um corpo de resposta (`body`) vazio ou com valor `null`. Propõe-se que a API passe a retornar o **objeto completo que foi recém-criado**, incluindo o seu identificador único (ID/Key) e uma mensagem descritiva de confirmação.

2. Cenário Atual vs. Proposto

- **Estado Atual:** - Status: `201 Created`
  - Body: `null` ou `{}`
  - *Impacto:* O Front-end não sabe qual foi o ID gerado para o novo item sem realizar uma nova busca ou atualizar a página inteira.
- **Estado Proposto:**
  - Status: `201 Created`
  - Body: `{ "message": "Livro cadastrado com sucesso", "data": { "id": "123", "title": "Dom Casmurro", ... } }`

    - *Impacto:* O Front-end pode injetar o novo item diretamente na lista em tela, economizando tráfego de rede e tornando a interface muito mais rápida.

3. Justificação Técnica e de Negócio

- **Performance (Redução de Latência):** Evita a necessidade de uma segunda chamada de rede (`GET`) imediatamente após o `POST` para obter os dados do objeto.
- **Consistência de Estado:** Garante que o Front-end e o Back-end estejam sincronizados com as informações exatas que foram persistidas no banco de dados (especialmente útil se houver campos preenchidos automaticamente pelo servidor, como datas de criação).
- **Boas Práticas REST:** Seguir o padrão de maturidade REST, onde a resposta de uma criação deve fornecer o recurso criado ou o link para acessá-lo.

4. Sugestão de Implementação

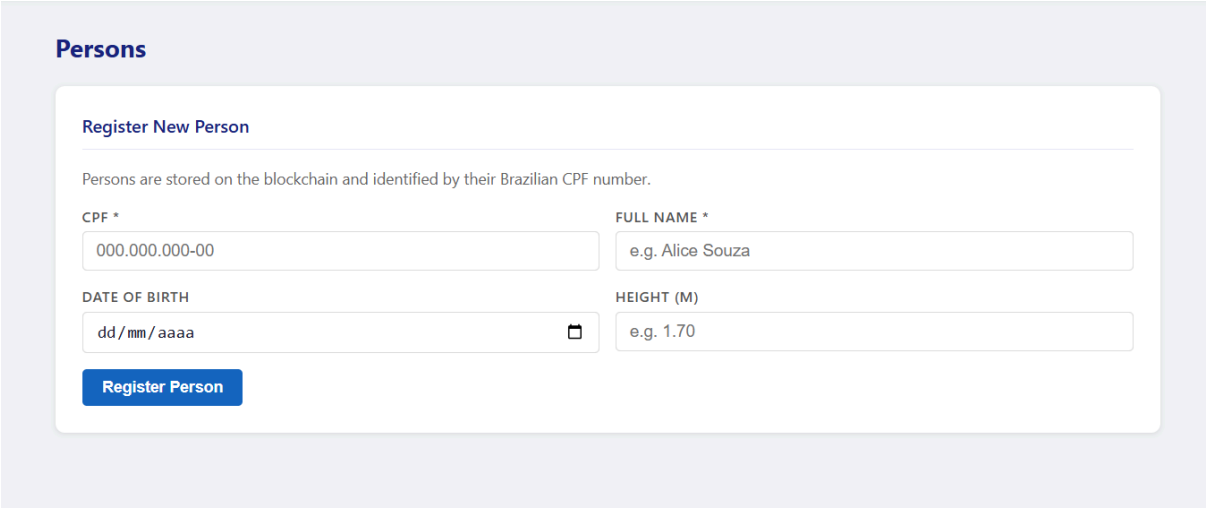
- **Back-end:** Alterar a lógica do controlador (`controller`) para que, após a persistência bem-sucedida, o objeto retornado pelo banco de dados seja encapsulado no corpo da resposta HTTP.
- **Padronização:** Criar um invólucro (Wrapper) padrão de resposta para todas as rotas de escrita, garantindo que o campo `"message"` sempre contenha uma explicação amigável do sucesso da operação.

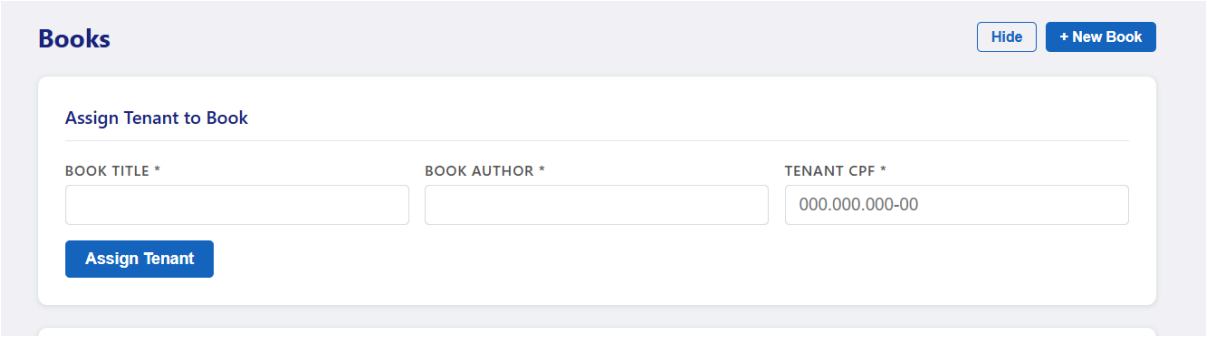
5. Critérios de Aceite para Validação (QA)

- Ao realizar um `POST` com sucesso, o corpo da resposta no Swagger e no Console do Navegador não deve ser `null`.
- A resposta deve conter o objeto completo com todos os seus atributos preenchidos.
- A mensagem de sucesso deve ser clara e condizente com a ação realizada (ex: "Biblioteca 'Central' criada com sucesso").

Proposta de Melhoria de Regra de Negócio e UI - Goledger Challenge

ID do Chamado: IMP-003 | **Prioridade:** Média (Refinamento de Requisito) **Módulo:** Gestão de Usuários e Listagem de Pessoas (Front-end / API) **Tipo:** Melhoria Funcional / UX **Relator:** Vitória (QA)





1. Descrição da Melhoria

Propõe-se a substituição dos campos de entrada de texto manual (Input Text) por componentes de **Seleção Inteligente (Autocomplete / Searchable Select)** nos campos "Book Title", "Book Author" e "Tenant CPF". Atualmente, o sistema exige que o usuário digite dados que já existem na base de dados, o que gera ineficiência e riscos de inconsistência.

2. Cenário Atual vs. Proposto

- **Estado Atual:** Os campos estão em branco e aceitam qualquer entrada de texto. Se o usuário digitar "O Senhor dos Aneis" (sem acento) em um livro cadastrado com acento, a atribuição pode falhar ou criar dados órfãos. Não há auxílio visual para saber quais livros ou pessoas estão disponíveis.

- **Estado Proposto:** Ao clicar no campo, deve ser aberta uma lista suspensa (Dropdown) com os itens já cadastrados no sistema. Conforme o usuário digita as primeiras letras, a lista deve filtrar automaticamente os resultados, permitindo apenas a seleção de itens válidos existentes no banco de dados.

3. Justificação Técnica e de Negócio

- **Prevenção de Erros (Error Proofing):** Elimina erros de digitação (typos) que invalidam o vínculo entre o livro e o usuário (Tenant).
- **Velocidade Operacional:** O administrador não precisa consultar outras telas ou planilhas para copiar o CPF exato ou o título do livro; a informação está a um clique de distância.
- **Integridade Referencial:** Garante que a API receba IDs válidos de entidades que realmente existem no sistema, evitando erros 400 (Bad Request) ou 404 (Not Found) desnecessários.

4. Sugestão de Implementação

- **Front-end:** Implementar componentes de Autocomplete (como *React Select* ou *MUI Autocomplete*). O campo "Tenant CPF" deve exibir o nome da pessoa ao lado do CPF na lista para facilitar a identificação.
- **Back-end/API:** Criação de endpoints de busca rápida (Lightweight Search) que retornem apenas o `ID` e o `Label` (nome/título) para otimizar o carregamento da lista suspensa.
- **UX/UI:** Adicionar um indicador visual de "carregando" (spinner) dentro do campo enquanto os dados são buscados na API.

5. Critérios de Aceite para Validação (QA)

- [ ] Ao clicar no campo "Book Title", o sistema deve exibir uma lista com os livros cadastrados.
- [ ] O sistema deve impedir a submissão do formulário com dados que não correspondam a itens da lista (Validação de Input).
- [ ] No campo "Tenant CPF", a busca deve funcionar tanto pelo número do CPF quanto pelo nome da pessoa vinculada.